MIAE 215 MIDTERM CHEATSHEET 2.0 · DR. GORDON OFFICIAL EXAM BLUEPRINT & PROBLEM GUIDE

CONCORDIA · GINA CODY SCHOOL · 1-SIDED FORMULA SHEET · 75 MIN · NO STRINGS

1. LIVE MIDTERM EXAM (FALL 2026 LIVE PAPER) — COMPLETE SOLUTIONS & PEDAGOGICAL TRACING TABLES

Official Paper Breakdown · Q1 (50%) + Q2 (50%) · Zero Blank Space

Q1: 6-PART 1D ARRAY COMPLETE PROGRAM WITH PEDAGOGICAL SOLUTION COMMENTS

50% WEIGHT

```cpp
#include <iostream>
#include <fstream>
using namespace std;
int main() {
  // a) Input 1D array A of 10 doubles from keyboard:
  double A[10];
  for (int i = 0; i < 10; i++) cin >> A[i];

  // b) Minimum element & index: CRITICAL TRAP -> MUST init minVal = A[0]!
  // (NEVER init to 0.0: fails if all numbers in array are strictly positive!)
  double minVal = A[0]; int minIdx = 0;
  for (int i = 1; i < 10; i++) {
    if (A[i] < minVal) { minVal = A[i]; minIdx = i; }
  }
  cout << "Min: A[" << minIdx << "] = " << minVal << "\n";

  // c) Range [-7, 7] verification: if all elements in range print message;
  // otherwise stop program immediately (use return 0 or exit(0)):
  bool inRange = true;
  for (int i = 0; i < 10; i++) {
    if (A[i] < -7.0 || A[i] > 7.0) { inRange = false; break; }
  }
  if (inRange) cout << "elements of A are in range\n";
  else return 0; // Immediate halt

  // d) Repeatedly input i1, i2 until valid (0..9 and i1 != i2):
  // Continuation condition (De Morgan): repeat while ANY condition is violated!
  int i1, i2;
  do {
    cin >> i1 >> i2;
  } while (i1 < 0 || i1 > 9 || i2 < 0 || i2 > 9 || i1 == i2);

  // e) Swap elements i1 and i2 of A: MUST use temp to prevent data overwrite!
  double temp = A[i1]; A[i1] = A[i2]; A[i2] = temp;

  // f) Save elements to "A.txt": open file stream, verify, write, close:
  ofstream fout("A.txt");
  if (!fout) return 1;
  for (int i = 0; i < 10; i++) fout << A[i] << "\n";
  fout.close();
  return 0;
}
```

Q2: SIGNATURE OUTPUT TRACING ENGINE, CODE SNIPPETS & EXECUTION TABLES

DR. GORDON TRAPS

- **1. Truncating Integer Div:** `1/(0+3) = 1/3 = 0` (NOT 0.33!). `7/2 = 3`. Float div: `7.0/2 = 3.5`.
- **2. IEEE-754 Div by Zero:** `7.0/(1-1) = 7.0/0.0 = inf` → **Does NOT crash!** Float div by zero yields **`inf`**.
- **3. Logical NOT `!i`: `!0 = true (1)`**; all non-zero numbers are true, so **`!1 = !2 = false (0)`**. **`else if (!i)`** executes ONLY at iteration *i=0*!
- **4. Array Left-Shift:** `B[i]=B[i+1]; i++;` sequentially shifts neighbor into current slot.
- **5. Uninitialized Garbage (G):** **`int x;`** holds undefined RAM garbage. **`x += B[i];`** stays Garbage! Write **"Garbage"** or **"G"** in booklet!
- **6. Loop Counter Retention & Bounds:** After **`for(i=0;i<5;i++)`**, *i* is **5**! In size-4 array, **`A[4]`** is out-of-bounds → **Garbage (G)**!
- **7. Pre-Break Trap:** Statements before **`if(cond) break;`** execute even on the terminating pass!
- **8. Modulo % Rules: `20 % 7 = 6`**. Sign follows LHS: **`-20 % 7 = -6`**; **`20 % -7 = +6`**.

Live Q2(a) Trace Table (double A[5]={2,3,5,7,11}, x=0.0): `Box: 0, inf, 7, 3.5, 4.5, 5`

i	A[i]	Branch Evaluated	x Expression & Value	Console Output
0	2	`else if (!i) → !0` is TRUE	`1 / (0 + 3) = 1 / 3 = 0` (int div truncates)	0
1	3	`else → 7.0 / (1 - 1)`	`7.0 / 0.0 = inf` (IEEE-754 double div)	inf
2	5	`else → 7.0 / (2 - 1)`	`7.0 / 1.0 = 7`	7
3	7	`else → 7.0 / (3 - 1)`	`7.0 / 2.0 = 3.5` (`7 > 7` is FALSE)	3.5
4	11	`if (A[i] > 7) → 11 > 7` TRUE	`x++ → 3.5 + 1 = 4.5`	4.5
post	–	Loop terminates: `5 < 5` is FALSE	Counter *i* retains failure value: `5`	5

Live Q2(b) Trace (int B[]={11,7,5,3,2,1}, i=0, x, a=20%7):

- *a* = 20 % 7 = **6**; *x* is uninitialized → **Garbage (G)**!
- Pass 1: B[0]=7 → {7,7,5,3,2,1}, i=1, x+=7 → **G**
- Pass 2: B[1]=5 → {7,5,5,3,2,1}, i=2, x+=5 → **G**
- Pass 3: B[2]=3 → {7,5,3,3,2,1}, i=3, x+=3 → **G**
- Pass 4: B[3]=2 → {7,5,3,2,2,1}, i=4, x+=2 → **G**
- Pass 5: B[4]=1 → {7,5,3,2,1,1}, i=5, x+=1 → **G**
- Pass 6: B[5]=1 ≥ 2 is **FALSE** → loop exits at *i=5*!
- Print: B[2]=**3**, B[3]=**2**, i=**5**, a=**6**, x=**G** → **3 2 5 6 G**

Past Exam Verified Box Keys:

F23 Q2a: 1 8 / 2 14 / 3 19 29 19 26 (pre-break!)

F23 Q2b: -9 / -9 / 7 / 7 / G (A[4] is past size 4 → Garbage G)

F22 Q2: 3 5 / 7 2 (modulo remainder)

Q1/Q2 Tracing: `y=x++` vs **`y=++x`**

2. ALL SIMILAR EXAM PROBLEMS & ALGORITHMS ACROSS PAST GORDON MIDTERMS & QUIZZES

Fall 2023 · Fall 2022 · Quiz 1 & 2 · Reviews 1 & 4 · Exact Twin Match

1. VALIDATED SIZE N & 2 ARRAYS

RESEMBLES Q1A

```cpp
int n; cin >> n;
while (n < 1 || n > 100) {
  cout << "Re-enter n (1-100): "; cin >> n;
}
double A[100], B[100];
for (int i = 0; i < n; i++) cin >> A[i];
for (int i = 0; i < n; i++) cin >> B[i];
```

2. MAX SEARCH & POS AVG

RESEMBLES Q1B

```cpp
// Max search (init maxVal = A[0]):
double maxV = A[0]; int maxIdx = 0;
for (int i = 1; i < n; i++)
  if (A[i] > maxV) { maxV = A[i]; maxIdx = i; }
// Average of positive elements only:
double s = 0; int cnt = 0;
for (int i = 0; i < n; i++)
  if (A[i] > 0) { s += A[i]; cnt++; }
double avg = cnt ? s / cnt : 0.0;
```

3. STRICT RANGE FILTER (0, PI)

RESEMBLES Q1C

```cpp
// Count elements strictly inside (0, PI):
const double PI = 4.0 * atan(1.0);
int count = 0;
for (int i = 0; i < n; i++) {
  if (B[i] > 0.0 && B[i] < PI) count++;
}
// Check all positive; if not exit immediately:
for (int i = 0; i < n; i++)
  if (B[i] <= 0.0) return 0;
```

4. MENU LOOP ALL-OPTION FLAG

RESEMBLES Q1D

```cpp
bool used[4] = { false }; int ch;
do {
  cout << "1=Avg, 2=Min, 3=Dot: "; cin >> ch;
  if (ch == 1) { /* avg */ used[1] = true; }
  else if (ch == 2) { /* min */ used[2] = true; }
  else if (ch == 3) { /* dot */ used[3] = true; }
} while (!(used[1] && used[2] && used[3]));
```

5. IN-PLACE ARRAY REVERSAL

RESEMBLES Q1E

```cpp
// Symmetrical swap i with n-1-i up to n/2:
for (int i = 0; i < n / 2; i++) {
  double temp = A[i];
  A[i] = A[n - 1 - i];
  A[n - 1 - i] = temp;
}
// Zero auxiliary RAM! Traverses only first half.
```

6. CYCLIC FORWARD SHIFT

RESEMBLES Q1E & Q2B

```cpp
// Shift forward: A[n-1] to 0, A[i] to i+1
double A_sh[100];
A_sh[0] = A[n - 1]; // Wrap last element
for (int i = 0; i < n - 1; i++) A_sh[i + 1] = A[i];
// In-place alternative (backward loop):
double last = A[n - 1];
for (int i = n - 1; i > 0; i--) A[i] = A[i - 1];
A[0] = last;
```

7. ARRAY MERGE / CONCATENATE

F22 Q1D

```cpp
// Merge A and B (size n) into C (size 2n):
double C[200];
for (int i = 0; i < n; i++) {
  C[i]     = A[i]; // 1st half: 0..n-1
  C[i + n] = B[i]; // 2nd half: n..2n-1
}
```

8. ARRAY EQUALITY TEST

MIDTERM REV 1

```cpp
bool equal = true;
for (int i = 0; i < n; i++) {
  // Doubles: if (fabs(A[i]-B[i]) > 1e-7)
  if (A[i] != B[i]) { equal = false; break; }
  // TRAP: NEVER add 'else equal=true;'!
}
if (equal) cout << "Equal"; else cout << "Not equal";
```

9. DOT PRODUCT & FREQ & NEW[]

Q2 / REV 4

```cpp
// Dot product: sum(A[i]*B[i]):
double dot = 0.0;
for (int i = 0; i < n; i++) dot += A[i] * B[i];

// Frequency count (val 1..7):
int freq[8] = { 0 }, v;
while (cin >> v && v >= 1 && v <= 7) freq[v]++;

// Dynamic array (zero leak):
double *p = new double[n]; delete[] p; p = nullptr;
```

3. ESSENTIAL LIBRARIES, NUMERICAL CONSTANTS & FUNCTION / FILE STREAM TEMPLATES

Directives & Memory Reference · No Fluff

DIRECTIVES & HEADERS

BOILERPLATE

`#include <iostream>` (cin, cout)

`#include <cmath>` (sqrt, pow, fabs, sin)

MATH & FORMATTING

FORMULAS

Concordia PI Constant:

`const double PI = 4.0 * atan(1.0);`

OPERATOR PRECEDENCE

ORDER

1. `() [] x++ x--`
2. `++x --x ! (type) sizeof` (R-to-L)

FUNCTIONS & FILE STREAMS

HIGH YIELD

Pass-by-Reference Swap (Modifies Caller):

```cpp
void swap(double &a, double &b){ double
```